```
      minNode ← findMin(node.right, node.level)
                #5
      newRight ← remove(node.right, minNode.point)
#6
      return new KdNode(minNode.point, node.left, newRight,
node.level)       #7
    elsif node.left != null
then                                                  #8
      minNode ← findMin(node.left, node.level)
        #9
      newRight ← remove(node.left,
minNode.point)                        #10
      return new KdNode(minNode.point, null, newRight,
node.level)            #11
    else
      return null
                          #12
  elsif compare(point, node) < 0
then                                             #13
    node.left ← remove(node.left, point)
    return node

else
               #14
    node.right ← remove(node.right, point)
    return node
```

#1 Remove a point from the tree rooted at `node`. The method will return the root of the tree after completing the operation.

#2 If node is `null`, we are traversing an empty tree, so can conclude the target point is not on the tree.

#3 If the node matches the point to remove, we can stop traversing the tree, and we need to remove current node.

#4 In case current node has a right child, we are in the most generic situation, so: